SwinUnet Wildfire Training & Test (Colab)

This notebook trains SwinUnet on a single fold of the WildfireSpreadTS HDF5 dataset, then evaluates on the held-out test split and reports Average Precision (AP) and F1.

Prerequisites:

- HDF5 dataset already on Google Drive (from `download_and_convert_dataset.ipynb`)
- **Runtime → Change runtime type → GPU or TPU** (T4/V100 GPU or TPU v2/v3 recommended)
- A GitHub Personal Access Token stored as a Colab secret named `GITHUB_TOKEN` (Colab sidebar → key icon → Add new secret)

1. Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

2. Configuration (user-editable)

```
REPO_ORG   = "amindell11"   # Replace with your GitHub username or organisation
REPO_NAME  = "wildfire-ts-swin"
HDF5_DIR   = "/content/drive/MyDrive/wildfire_dataset/hdf5"
OUTPUT_DIR = "/content/drive/MyDrive/wildfire_runs/fold0"

DATA_FOLD_ID             = 0      # 0-11; which train/val/test year split
N_LEADING_OBSERVATIONS   = 1      # 1 or 5
MAX_EPOCHS               = 100
BATCH_SIZE               = 16
BASE_LR                  = 1e-4
FOCAL_GAMMA              = 2.0
CROP_SIDE_LENGTH         = 128
SEED                     = 42
NUM_WORKERS              = 4      # parallel data workers; 4 is safe on Colab

# Checkpointing
CHECKPOINT_INTERVAL = 10          # save a full training state every N epochs
RESUME_CHECKPOINT   = None        # set to a path to resume, e.g.:
                                  #   f"{OUTPUT_DIR}/checkpoint_epoch049.pth"
```

2b. Copy HDF5 files to local disk

Google Drive I/O is ~100× slower than local NVMe. Copying once here keeps the A100 busy instead of waiting on Drive reads. Takes 1–3 min depending on dataset size; skips files already copied.

```
import os, shutil, time
from concurrent.futures import ThreadPoolExecutor, as_completed

_LOCAL_HDF5 = "/content/hdf5"
_COPY_WORKERS = 8  # parallel Drive→local transfers

# Collect all (src, dst) pairs
_pairs = []
for _root, _dirs, _files in os.walk(HDF5_DIR):
    _rel = os.path.relpath(_root, HDF5_DIR)
    _dst_dir = os.path.join(_LOCAL_HDF5, _rel)
    os.makedirs(_dst_dir, exist_ok=True)
    for _f in _files:
        _src = os.path.join(_root, _f)
        _dst = os.path.join(_dst_dir, _f)
        if not os.path.exists(_dst):
            _pairs.append((_src, _dst))

if not _pairs:
    print(f"All files already present at {_LOCAL_HDF5}")
else:
    print(f"Copying {len(_pairs)} files with {_COPY_WORKERS} parallel workers...")
    _t0 = time.time()
    _done = 0
    with ThreadPoolExecutor(max_workers=_COPY_WORKERS) as _pool:
        _futs = {_pool.submit(shutil.copy2, s, d): (s, d) for s, d in _pairs}
        for _fut in as_completed(_futs):
            _fut.result()  # re-raises any exception
```

```
            _done += 1
            if _done % 20 == 0 or _done == len(_pairs):
                print(f"  {_done}/{len(_pairs)} files copied  ({time.time()-_t0:.0f}s elapsed)")
    print(f"Done in {time.time() - _t0:.0f}s")

HDF5_DIR = _LOCAL_HDF5  # all subsequent cells read from local disk
print(f"HDF5_DIR → {HDF5_DIR}")
```

```
Copying 607 files with 8 parallel workers...
  20/607 files copied  (79s elapsed)
  40/607 files copied  (94s elapsed)
  60/607 files copied  (128s elapsed)
  80/607 files copied  (132s elapsed)
  100/607 files copied  (149s elapsed)
  120/607 files copied  (173s elapsed)
  140/607 files copied  (179s elapsed)
  160/607 files copied  (193s elapsed)
  180/607 files copied  (215s elapsed)
  200/607 files copied  (238s elapsed)
  220/607 files copied  (260s elapsed)
  240/607 files copied  (289s elapsed)
  260/607 files copied  (308s elapsed)
  280/607 files copied  (326s elapsed)
  300/607 files copied  (354s elapsed)
  320/607 files copied  (374s elapsed)
  340/607 files copied  (404s elapsed)
  360/607 files copied  (427s elapsed)
  380/607 files copied  (456s elapsed)
  400/607 files copied  (479s elapsed)
  420/607 files copied  (509s elapsed)
  440/607 files copied  (531s elapsed)
  460/607 files copied  (557s elapsed)
  480/607 files copied  (579s elapsed)
  500/607 files copied  (606s elapsed)
  520/607 files copied  (639s elapsed)
  540/607 files copied  (671s elapsed)
  560/607 files copied  (703s elapsed)
  580/607 files copied  (737s elapsed)
  600/607 files copied  (767s elapsed)
  607/607 files copied  (769s elapsed)
Done in 769s
HDF5_DIR → /content/hdf5
```

3. Clone repo and install dependencies

```
from google.colab import userdata
_repo_url = f"https://github.com/amindell11/wildfire-ts-swin.git"
!rm -rf /content/wildfire-ts-swin
!git clone $_repo_url /content/wildfire-ts-swin
!pip install -q -r /content/wildfire-ts-swin/requirements.txt
!git -C /content/wildfire-ts-swin log --oneline
```

```
Cloning into '/content/wildfire-ts-swin'...
remote: Enumerating objects: 119, done.
remote: Counting objects: 100% (119/119), done.
remote: Compressing objects: 100% (83/83), done.
remote: Total 119 (delta 52), reused 101 (delta 34), pack-reused 0 (from 0)
Receiving objects: 100% (119/119), 166.79 KiB | 3.03 MiB/s, done.
Resolving deltas: 100% (52/52), done.
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 87.2/87.2 kB 3.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 857.3/857.3 kB 22.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 76.7/76.7 kB 11.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 983.2/983.2 kB 75.2 MB/s eta 0:00:00
4dc81c7 (HEAD -> main, origin/main, origin/HEAD) Implement checkpointing functionality in trainer_wildfire and update Colab note
b6c71b5 restore tqdm bars; keep parallel HDF5 copy
2f4a549 parallel HDF5 copy; plain print logging for VS Code Colab plugin
0d1973c add persistent_workers to suppress DataLoader GC error in Jupyter
83975c6 fix console logging to always show val metrics via tqdm.write
ba2df2e fix HDF5 copy to handle year subdirectories
91fc34b copy HDF5 to local disk before training to avoid Drive I/O bottleneck
4400505 use tqdm.auto for proper Jupyter/Colab rendering; device-agnostic notebook
d3df338 update log clarity
62ac090 fixes
fc59794 add support for TPU / XLA
894aeda add notebooks
b9fc63a add init.py to recognize datasets as a packages
2e898e2 feat: add AP val and Focal loss for swin training
4aaa44f feat: Average Precision for model evaluation
b63951d update README.md
fd11e4d add code for loading wildfire data
3071415 add swin-unet base code
eef6f9e Initial commit
```

```
import sys
sys.path.insert(0, f'/content/{REPO_NAME}')
```

4. Detect accelerator (GPU / TPU / CPU)

```
!nvidia-smi
```

```
Wed Feb 25 22:38:52 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA RTX PRO 6000 Blac...    Off |   00000000:05:00.0 Off |                    0 |
| N/A   25C    P8             28W /  600W |       0MiB /  97887MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
import torch

# Detect the best available device: TPU (XLA) > GPU (CUDA) > CPU
try:
    import torch_xla.core.xla_model as xm
    DEVICE = xm.xla_device()
    print(f"TPU device detected: {DEVICE}")
except ImportError:
    if torch.cuda.is_available():
        DEVICE = torch.device('cuda')
        print(f"GPU: {torch.cuda.get_device_name(0)}")
    else:
        DEVICE = torch.device('cpu')
        print("WARNING: No GPU or TPU detected — running on CPU (will be slow).")
```

```
GPU: NVIDIA RTX PRO 6000 Blackwell Server Edition
```

```
## 5a. Launch TensorBoard (run this before training; refresh the board while training runs)
import os
os.makedirs(f"{OUTPUT_DIR}/log", exist_ok=True)
%load_ext tensorboard
%tensorboard --logdir {OUTPUT_DIR}/log
```

TensorBoard INACTIVE

No dashboards are active for the current data set.

Probable causes:

- You haven't written any data to your event files.
- TensorBoard can't find your event files.

If you're new to using TensorBoard, and want to find out how to add data and set up your event files, check out the README and perhaps the TensorBoard tutorial.

If you think TensorBoard is configured properly, please see the section of the README devoted to missing data problems and consider filing an issue on GitHub.

Last reload: Mar 9, 2026, 10:08:55 AM

Log directory: /content/drive/MyDrive/wildfire_runs/fold0/log

5. Train

```
import os
import random
import types
import numpy as np
import torch
import torch.backends.cudnn as cudnn

from config import get_config
from networks.vision_transformer import SwinUnet
from trainer_wildfire import trainer_wildfire
from datasets.wildfire import N_FEATURES_PER_TIMESTEP

if torch.cuda.is_available():
    cudnn.benchmark = True
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)

in_chans = N_LEADING_OBSERVATIONS * N_FEATURES_PER_TIMESTEP
extra_opts = [
    'MODEL.SWIN.IN_CHANS', str(in_chans),
    'MODEL.PRETRAIN_CKPT', 'None',
]

args = types.SimpleNamespace(
    data_dir=HDF5_DIR,
    output_dir=OUTPUT_DIR,
    n_leading_observations=N_LEADING_OBSERVATIONS,
    n_leading_observations_test_adjustment=N_LEADING_OBSERVATIONS,
    crop_side_length=CROP_SIDE_LENGTH,
    load_from_hdf5=True,
    data_fold_id=DATA_FOLD_ID,
    max_epochs=MAX_EPOCHS,
```

```
    batch_size=BATCH_SIZE,
    base_lr=BASE_LR,
    num_workers=NUM_WORKERS,
    eval_interval=1,
    seed=SEED,
    n_gpu=1,
    focal_gamma=FOCAL_GAMMA,
    cfg=f'/content/{REPO_NAME}/configs/swin_tiny_patch4_window4_128_wildfire.yaml',
    opts=extra_opts,
    zip=False,
    cache_mode='part',
    resume=RESUME_CHECKPOINT,          # set in config cell to restart from a checkpoint
    checkpoint_interval=CHECKPOINT_INTERVAL,
    accumulation_steps=None,
    use_checkpoint=False,
    amp_opt_level='O1',
    tag=None,
    eval=False,
    throughput=False,
)

config = get_config(args)
os.makedirs(OUTPUT_DIR, exist_ok=True)

net = SwinUnet(config, img_size=config.DATA.IMG_SIZE, num_classes=2).to(DEVICE)
print(f"Model in_chans={in_chans}  (n_leading_observations={N_LEADING_OBSERVATIONS} × 40 features)")
print(f"Model parameters: {sum(p.numel() for p in net.parameters()) / 1e6:.1f}M")
```

```
/usr/local/lib/python3.12/dist-packages/timm/models/layers/__init__.py:49: FutureWarning: Importing from timm.models.layers is d
  warnings.warn(f"Importing from {__name__} is deprecated, please import via timm.layers", FutureWarning)
=> merge config from /content/wildfire-ts-swin/configs/swin_tiny_patch4_window4_128_wildfire.yaml
SwinTransformerSys expand initial----depths:[2, 2, 2, 2];depths_decoder:[1, 2, 2, 2];drop_path_rate:0.1;num_classes:2
/usr/local/lib/python3.12/dist-packages/torch/functional.py:505: UserWarning: torch.meshgrid: in an upcoming release, it will be
  return _VF.meshgrid(tensors, **kwargs)  # type: ignore[attr-defined]
---final upsample expand_first---
Model in_chans=40  (n_leading_observations=1 × 40 features)
Model parameters: 27.2M
```

5a. Load Weights from Drive (optional)

Run this cell to manually load model weights into `net` from any saved file on Drive.

- Use **`best_model.pth`** or a **`checkpoint_epoch*.pth`** path.
- A full `checkpoint_epoch*.pth` file contains `model_state`, `optimizer_state`, `scheduler_state`, `iter_num`, and `best_val_ap`. The trainer will restore all of these automatically when `RESUME_CHECKPOINT` is set in the config cell above.
- Use this cell instead when you only want to swap the model weights (e.g. for evaluation) without touching the training state.

```
LOAD_WEIGHTS_FROM_PATH = None  # e.g. f"{OUTPUT_DIR}/best_model.pth"
                               #      f"{OUTPUT_DIR}/checkpoint_epoch049.pth"

if LOAD_WEIGHTS_FROM_PATH is not None:
    state = torch.load(LOAD_WEIGHTS_FROM_PATH, map_location=DEVICE)
    # Full checkpoint dicts store weights under 'model_state'; bare state dicts work too.
    if isinstance(state, dict) and 'model_state' in state:
        state = state['model_state']
    if next(iter(state)).startswith('module.'):
        state = {k[len('module.'):]: v for k, v in state.items()}
    net.load_state_dict(state)
    print(f"Weights loaded from: {LOAD_WEIGHTS_FROM_PATH}")
else:
    print("Skipped (LOAD_WEIGHTS_FROM_PATH is None).")
```

5b. Run Training

Runs the training loop.

- **Fresh run:** leave `RESUME_CHECKPOINT = None` in the config cell and just execute this cell.
- **Resume after interruption:** set `RESUME_CHECKPOINT` to a `checkpoint_epoch*.pth` path in the config cell, re-run the model-init cell above, then run this cell.
- Periodic checkpoints (`checkpoint_epoch{N:03d}.pth`) are written to `OUTPUT_DIR` every `CHECKPOINT_INTERVAL` epochs and include full optimizer + scheduler state.

```
trainer_wildfire(args, net, OUTPUT_DIR, device=DEVICE)
```

```
Epoch 005  val_loss=0.0884  AP=0.3438  F1=0.2045  Prec=0.1173  Rec=0.7948
```

```
Epoch 006  train_loss=0.0609
```

```
Epoch 006  val_loss=0.0843  AP=0.3552  F1=0.1958  Prec=0.1114  Rec=0.8087 <-- new best
  -> New best AP=0.3552, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 007  train_loss=0.0603
```

```
Epoch 007  val_loss=0.0848  AP=0.3614  F1=0.2318  Prec=0.1361  Rec=0.7815 <-- new best
  -> New best AP=0.3614, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 008  train_loss=0.0590
```

```
Epoch 008  val_loss=0.0907  AP=0.3658  F1=0.3141  Prec=0.2007  Rec=0.7227 <-- new best
  -> New best AP=0.3658, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 009  train_loss=0.0581
```

```
Epoch 009  train_loss=0.0581
  -> Periodic checkpoint saved: /content/drive/MyDrive/wildfire_runs/fold0/checkpoint_epoch009.pth
```

```
Epoch 009  val_loss=0.0846  AP=0.3670  F1=0.1887  Prec=0.1067  Rec=0.8152 <-- new best
  -> New best AP=0.3670, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 010  train_loss=0.0579
```

```
Epoch 010  val_loss=0.0839  AP=0.3709  F1=0.1813  Prec=0.1019  Rec=0.8212 <-- new best
  -> New best AP=0.3709, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 011  train_loss=0.0582
```

```
Epoch 011  val_loss=0.0907  AP=0.3683  F1=0.1400  Prec=0.0763  Rec=0.8440
```

```
Epoch 012  train_loss=0.0570
```

```
Epoch 012  val_loss=0.0856  AP=0.3797  F1=0.2010  Prec=0.1147  Rec=0.8089 <-- new best
  -> New best AP=0.3797, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 013  train_loss=0.0567
```

```
Epoch 013  val_loss=0.0901  AP=0.3767  F1=0.2441  Prec=0.1449  Rec=0.7749
```

```
Epoch 014  train_loss=0.0564
```

```
Epoch 014  val_loss=0.0817  AP=0.3729  F1=0.2295  Prec=0.1345  Rec=0.7818
```

```
Epoch 015  train_loss=0.0554
```

```
Epoch 015  val_loss=0.0840  AP=0.3777  F1=0.2592  Prec=0.1556  Rec=0.7743
```

```
Epoch 016  train_loss=0.0548
```

```
Epoch 016  val_loss=0.0813  AP=0.3832  F1=0.2164  Prec=0.1250  Rec=0.8033 <-- new best
  -> New best AP=0.3832, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 017  train_loss=0.0540
```

```
Epoch 017  val_loss=0.0826  AP=0.3866  F1=0.2303  Prec=0.1346  Rec=0.7965 <-- new best
  -> New best AP=0.3866, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 018  train_loss=0.0550
```

```
Epoch 018  val_loss=0.0793  AP=0.3885  F1=0.2136  Prec=0.1229  Rec=0.8159 <-- new best
  -> New best AP=0.3885, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 019  train_loss=0.0548
  -> Periodic checkpoint saved: /content/drive/MyDrive/wildfire_runs/fold0/checkpoint_epoch019.pth
```

```
Epoch 019  val_loss=0.0825  AP=0.3900  F1=0.1955  Prec=0.1110  Rec=0.8197 <-- new best
  -> New best AP=0.3900, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 020  train_loss=0.0533
```

```
Epoch 020  val_loss=0.0925  AP=0.3733  F1=0.1251  Prec=0.0675  Rec=0.8436
```

```
Epoch 021  train_loss=0.0539
```

```
Epoch 021  val_loss=0.0790  AP=0.3908  F1=0.2071  Prec=0.1185  Rec=0.8202 <-- new best
  -> New best AP=0.3908, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 022  train_loss=0.0538
```

```
Epoch 022  val_loss=0.0790  AP=0.3837  F1=0.2311  Prec=0.1349  Rec=0.8077
```

```
Epoch 023  train_loss=0.0535
```

```
Epoch 023  val_loss=0.0803  AP=0.3891  F1=0.2071  Prec=0.1185  Rec=0.8197
```

```
Epoch 024  train_loss=0.0522
```

```
Epoch 024  val_loss=0.0832  AP=0.3980  F1=0.2597  Prec=0.1556  Rec=0.7843 <-- new best
  -> New best AP=0.3980, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 025  train_loss=0.0527
```

```
Epoch 025  val_loss=0.0792  AP=0.4029  F1=0.2273  Prec=0.1322  Rec=0.8102 <-- new best
  -> New best AP=0.4029, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 026  train_loss=0.0527
```

```
Epoch 026  val_loss=0.0822  AP=0.3966  F1=0.1935  Prec=0.1096  Rec=0.8257
```

```
Epoch 027  train_loss=0.0525
```

```
Epoch 027  val_loss=0.0808  AP=0.3846  F1=0.1957  Prec=0.1111  Rec=0.8219
```

```
Epoch 028  train_loss=0.0517
```

```
Epoch 028  val_loss=0.0801  AP=0.3920  F1=0.2048  Prec=0.1169  Rec=0.8241
```

```
Epoch 029  train_loss=0.0522
  -> Periodic checkpoint saved: /content/drive/MyDrive/wildfire_runs/fold0/checkpoint_epoch029.pth
```

```
Epoch 029  val_loss=0.0815  AP=0.3816  F1=0.2584  Prec=0.1546  Rec=0.7879
```

```
Epoch 030  train_loss=0.0511
```

```
Epoch 030  val_loss=0.0784  AP=0.3942  F1=0.1974  Prec=0.1120  Rec=0.8344
```

```
Epoch 031  train_loss=0.0514
```

```
Epoch 031  val_loss=0.0837  AP=0.3970  F1=0.1931  Prec=0.1094  Rec=0.8262
```

```
Epoch 032  train_loss=0.0502
```

```
Epoch 032  val_loss=0.0800  AP=0.4028  F1=0.2706  Prec=0.1632  Rec=0.7916
```

```
Epoch 033  train_loss=0.0521
```

```
Epoch 033  val_loss=0.0785  AP=0.4031  F1=0.2464  Prec=0.1454  Rec=0.8068 <-- new best
  -> New best AP=0.4031, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 034  train_loss=0.0523
```

```
Epoch 034  val_loss=0.0845  AP=0.3984  F1=0.2663  Prec=0.1601  Rec=0.7916
```

```
Epoch 035  train_loss=0.0506

Epoch 035  val_loss=0.0820  AP=0.3979  F1=0.2353  Prec=0.1379  Rec=0.8016

Epoch 036  train_loss=0.0501

Epoch 036  val_loss=0.0788  AP=0.4048  F1=0.2572  Prec=0.1531  Rec=0.8028 <-- new best
  -> New best AP=0.4048, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth

Epoch 037  train_loss=0.0500

Epoch 037  val_loss=0.0777  AP=0.4058  F1=0.2164  Prec=0.1243  Rec=0.8336 <-- new best
  -> New best AP=0.4058, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth

Epoch 038  train_loss=0.0499

Epoch 038  val_loss=0.0779  AP=0.4066  F1=0.2195  Prec=0.1265  Rec=0.8273 <-- new best
  -> New best AP=0.4066, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth

Epoch 039  train_loss=0.0506
  -> Periodic checkpoint saved: /content/drive/MyDrive/wildfire_runs/fold0/checkpoint_epoch039.pth

Epoch 039  val_loss=0.0821  AP=0.4129  F1=0.3012  Prec=0.1871  Rec=0.7707 <-- new best
  -> New best AP=0.4129, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 040  train_loss=0.0492
```

```
Epoch 040  val_loss=0.0758  AP=0.4171  F1=0.2207  Prec=0.1271  Rec=0.8364 <-- new best
  -> New best AP=0.4171, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 041  train_loss=0.0489
```

```
Epoch 041  val_loss=0.0769  AP=0.4124  F1=0.2060  Prec=0.1173  Rec=0.8446
```

```
Epoch 042  train_loss=0.0492
```

```
Epoch 042  val_loss=0.0786  AP=0.4193  F1=0.2539  Prec=0.1508  Rec=0.8027 <-- new best
  -> New best AP=0.4193, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 043  train_loss=0.0493
```

```
Epoch 043  val_loss=0.0750  AP=0.4099  F1=0.2162  Prec=0.1242  Rec=0.8346
```

```
Epoch 044  train_loss=0.0494
```

```
Epoch 044  val_loss=0.0797  AP=0.4135  F1=0.2841  Prec=0.1731  Rec=0.7916
```

```
Epoch 045  train_loss=0.0485
```

```
Epoch 045  val_loss=0.0799  AP=0.4099  F1=0.2934  Prec=0.1806  Rec=0.7820
```

```
Epoch 046  train_loss=0.0482
```

```
Epoch 046  val_loss=0.0773  AP=0.4159  F1=0.2817  Prec=0.1714  Rec=0.7917
```

```
Epoch 047  train_loss=0.0479
```

```
Epoch 047  val_loss=0.0762  AP=0.4231  F1=0.2456  Prec=0.1444  Rec=0.8217 <-- new best
  -> New best AP=0.4231, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 048  train_loss=0.0478
```

```
Epoch 048  val_loss=0.0812  AP=0.4094  F1=0.2602  Prec=0.1553  Rec=0.8013
```

```
Epoch 049  train_loss=0.0475
  -> Periodic checkpoint saved: /content/drive/MyDrive/wildfire_runs/fold0/checkpoint_epoch049.pth
```

```
Epoch 049  val_loss=0.0757  AP=0.4233  F1=0.2808  Prec=0.1704  Rec=0.7974 <-- new best
  -> New best AP=0.4233, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 050  train_loss=0.0484
```

```
Epoch 050  val_loss=0.0746  AP=0.4228  F1=0.2228  Prec=0.1286  Rec=0.8340
```

```
Epoch 051  train_loss=0.0478
```

```
Epoch 051  val_loss=0.0794  AP=0.4230  F1=0.2855  Prec=0.1742  Rec=0.7903
```

```
Epoch 052  train_loss=0.0466
```

```
Epoch 052  val_loss=0.0757  AP=0.4313  F1=0.2274  Prec=0.1315  Rec=0.8374 <-- new best
  -> New best AP=0.4313, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 053  train_loss=0.0474
```

```
Epoch 053  val_loss=0.0750  AP=0.4299  F1=0.2490  Prec=0.1467  Rec=0.8243
```

```
Epoch 054  train_loss=0.0484
```

```
Epoch 054  val_loss=0.0755  AP=0.4276  F1=0.2559  Prec=0.1520  Rec=0.8084
```

```
Epoch 055  train_loss=0.0470

Epoch 055  val_loss=0.0774  AP=0.4257  F1=0.2601  Prec=0.1549  Rec=0.8096

Epoch 056  train_loss=0.0472

Epoch 056  val_loss=0.0765  AP=0.4285  F1=0.2262  Prec=0.1309  Rec=0.8316

Epoch 057  train_loss=0.0477

Epoch 057  val_loss=0.0807  AP=0.4225  F1=0.2453  Prec=0.1445  Rec=0.8097

Epoch 058  train_loss=0.0473

Epoch 058  val_loss=0.0768  AP=0.4266  F1=0.2494  Prec=0.1469  Rec=0.8237

Epoch 059  train_loss=0.0464
  -> Periodic checkpoint saved: /content/drive/MyDrive/wildfire_runs/fold0/checkpoint_epoch059.pth

Epoch 059  val_loss=0.0769  AP=0.4295  F1=0.2554  Prec=0.1511  Rec=0.8241

Epoch 060  train_loss=0.0463
```

```
Epoch 060  val_loss=0.0786  AP=0.4280  F1=0.2781  Prec=0.1684  Rec=0.7970
```

```
Epoch 061  train_loss=0.0483
```

```
Epoch 061  val_loss=0.0760  AP=0.4240  F1=0.1868  Prec=0.1048  Rec=0.8591
```

```
Epoch 062  train_loss=0.0465
```

```
Epoch 062  val_loss=0.0780  AP=0.4273  F1=0.2552  Prec=0.1515  Rec=0.8088
```

```
Epoch 063  train_loss=0.0465
```

```
Epoch 063  val_loss=0.0777  AP=0.4287  F1=0.2409  Prec=0.1410  Rec=0.8274
```

```
Epoch 064  train_loss=0.0460
```

```
Epoch 064  val_loss=0.0792  AP=0.4271  F1=0.2591  Prec=0.1542  Rec=0.8102
```

```
Epoch 065  train_loss=0.0455
```

```
Epoch 065  val_loss=0.0786  AP=0.4293  F1=0.2626  Prec=0.1566  Rec=0.8125
Epoch 066  train_loss=0.0448
Epoch 066  val_loss=0.0787  AP=0.4295  F1=0.2640  Prec=0.1577  Rec=0.8090
Epoch 067  train_loss=0.0452
Epoch 067  val_loss=0.0793  AP=0.4297  F1=0.2588  Prec=0.1539  Rec=0.8119
Epoch 068  train_loss=0.0455
Epoch 068  val_loss=0.0797  AP=0.4345  F1=0.2729  Prec=0.1643  Rec=0.8063 <-- new best
  -> New best AP=0.4345, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
Epoch 069  train_loss=0.0449
  -> Periodic checkpoint saved: /content/drive/MyDrive/wildfire_runs/fold0/checkpoint_epoch069.pth
Epoch 069  val_loss=0.0772  AP=0.4349  F1=0.2855  Prec=0.1739  Rec=0.7974 <-- new best
  -> New best AP=0.4349, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
Epoch 070  train_loss=0.0448
Epoch 070  val_loss=0.0769  AP=0.4338  F1=0.2635  Prec=0.1572  Rec=0.8126
```

```
Epoch 071  train_loss=0.0449

Epoch 071  val_loss=0.0758  AP=0.4282  F1=0.2411  Prec=0.1412  Rec=0.8250

Epoch 072  train_loss=0.0448

Epoch 072  val_loss=0.0821  AP=0.4305  F1=0.2540  Prec=0.1505  Rec=0.8139

Epoch 073  train_loss=0.0443

Epoch 073  val_loss=0.0777  AP=0.4299  F1=0.2567  Prec=0.1522  Rec=0.8195

Epoch 074  train_loss=0.0442

Epoch 074  val_loss=0.0783  AP=0.4328  F1=0.2700  Prec=0.1620  Rec=0.8089

Epoch 075  train_loss=0.0444

Epoch 075  val_loss=0.0817  AP=0.4286  F1=0.2689  Prec=0.1616  Rec=0.8001
```

```
Epoch 076  train_loss=0.0439
```

```
Epoch 076  val_loss=0.0784  AP=0.4306  F1=0.2368  Prec=0.1382  Rec=0.8271
```

```
Epoch 077  train_loss=0.0437
```

```
Epoch 077  val_loss=0.0778  AP=0.4361  F1=0.2680  Prec=0.1607  Rec=0.8080 <-- new best
  -> New best AP=0.4361, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 078  train_loss=0.0436
```

```
Epoch 078  val_loss=0.0775  AP=0.4359  F1=0.2516  Prec=0.1485  Rec=0.8226
```

```
Epoch 079  train_loss=0.0443
  -> Periodic checkpoint saved: /content/drive/MyDrive/wildfire_runs/fold0/checkpoint_epoch079.pth
```

```
Epoch 079  val_loss=0.0784  AP=0.4359  F1=0.2783  Prec=0.1682  Rec=0.8044
```

```
Epoch 080  train_loss=0.0442
```

```
Epoch 080  val_loss=0.0809  AP=0.4285  F1=0.2574  Prec=0.1530  Rec=0.8115
```

```
Epoch 081  train_loss=0.0439
```

```
Epoch 081  val_loss=0.0803  AP=0.4280  F1=0.2596  Prec=0.1545  Rec=0.8102
```

```
Epoch 082  train_loss=0.0437
```

```
Epoch 082  val_loss=0.0827  AP=0.4386  F1=0.2971  Prec=0.1829  Rec=0.7901 <-- new best
  -> New best AP=0.4386, saved to /content/drive/MyDrive/wildfire_runs/fold0/best_model.pth
```

```
Epoch 083  train_loss=0.0436
```

```
Epoch 083  val_loss=0.0785  AP=0.4353  F1=0.2502  Prec=0.1476  Rec=0.8207
```

```
Epoch 084  train_loss=0.0440
```

```
Epoch 084  val_loss=0.0791  AP=0.4344  F1=0.2806  Prec=0.1700  Rec=0.8015
```

```
Epoch 085  train_loss=0.0432
```

```
Epoch 085  val_loss=0.0782  AP=0.4322  F1=0.2517  Prec=0.1487  Rec=0.8193
```

```
Epoch 086  train_loss=0.0424
```

```
Epoch 086  val_loss=0.0793  AP=0.4343  F1=0.2687  Prec=0.1611  Rec=0.8090
```

```
Epoch 087  train_loss=0.0432
```

```
Epoch 087  val_loss=0.0795  AP=0.4345  F1=0.2604  Prec=0.1550  Rec=0.8134
```

```
Epoch 088  train_loss=0.0427
```

```
Epoch 088  val_loss=0.0803  AP=0.4347  F1=0.2617  Prec=0.1559  Rec=0.8145
```

```
Epoch 089  train_loss=0.0427
  -> Periodic checkpoint saved: /content/drive/MyDrive/wildfire_runs/fold0/checkpoint_epoch089.pth
```

```
Epoch 089  val_loss=0.0803  AP=0.4348  F1=0.2662  Prec=0.1592  Rec=0.8114
```

```
Epoch 090  train_loss=0.0429
```

```
Epoch 090  val_loss=0.0789  AP=0.4328  F1=0.2540  Prec=0.1503  Rec=0.8187
```

```
Epoch 091  train_loss=0.0426
```

```
Epoch 091  val_loss=0.0801  AP=0.4341  F1=0.2617  Prec=0.1559  Rec=0.8133
```

```
Epoch 092  train_loss=0.0424
```

```
Epoch 092  val_loss=0.0797  AP=0.4332  F1=0.2537  Prec=0.1501  Rec=0.8194
```

```
Epoch 093  train_loss=0.0423
```

```
Epoch 093  val_loss=0.0808  AP=0.4312  F1=0.2547  Prec=0.1509  Rec=0.8160
```

```
Epoch 094  train_loss=0.0423
```

```
Epoch 094  val_loss=0.0799  AP=0.4346  F1=0.2578  Prec=0.1531  Rec=0.8157
```

```
Epoch 095  train_loss=0.0420
```

```
Epoch 095  val_loss=0.0794  AP=0.4339  F1=0.2567  Prec=0.1522  Rec=0.8176
```

```
Epoch 096  train_loss=0.0420
```

```
Epoch 096  val_loss=0.0797  AP=0.4326  F1=0.2625  Prec=0.1565  Rec=0.8145
```

6. Evaluate on test split

```
import numpy as np
from torch.utils.data import DataLoader
from tqdm import tqdm
from datasets.wildfire import WildfireDataset, get_year_split
from utils import compute_binary_metrics, compute_ap

train_years, val_years, test_years = get_year_split(DATA_FOLD_ID)

ckpt_path = f"{OUTPUT_DIR}/best_model.pth"
state_dict = torch.load(ckpt_path, map_location=DEVICE)
if list(state_dict.keys())[0].startswith('module.'):
    state_dict = {k.replace('module.', ''): v for k, v in state_dict.items()}
net.load_state_dict(state_dict)
net.eval()

db_test = WildfireDataset(
    data_dir=HDF5_DIR,
    included_fire_years=test_years,
    is_train=False,
    stats_years=train_years,
    n_leading_observations=N_LEADING_OBSERVATIONS,
    n_leading_observations_test_adjustment=N_LEADING_OBSERVATIONS,
    crop_side_length=CROP_SIDE_LENGTH,
    load_from_hdf5=True,
)

test_loader = DataLoader(
    db_test, batch_size=BATCH_SIZE, shuffle=False,
```